Listing 3.14 Using `format_html()` to mark data as safe in the Django REPL

```
>>> from django.utils.html import format_html
>>> instrument = "bass"
>>> instrument = format_html("big <i>{}</i>", instrument)
>>> data = {"instrument":instrument}
>>> t.render(Context(data))
"I love the sound a big <i>bass</i> makes.\n
➥ A shiny big <i>bass</i> is good.\n\n\n
➥ Don't put a big <i>bass</i> too close to your ear.\n\n"
```

> **User input and the safety of strings**
>
> Escaping strings becomes that much more important when you start dealing with user input. Malicious users could input data that intentionally damages your web pages. Be careful using any of the tools that turn off escaping, and make sure you know what you will be allowing on your page.
>
> There are even more string management utilities in `django.utils.html`, which deal with rendering JSON, doing tag processing, and addressing those pesky strings input by users. See the documentation for more details: http://mng.bz/7dZQ.

Escaping is all about the micro level of HTML: making sure your data gets rendered as you intended. At the macro level, you have a different problem: HTML is repetitive. The Django template language includes a way to compose and inherit blocks to help you write less HTML.

3.8 Composing templates out of blocks

Modern HTML pages tend to have a lot of moving parts, navigation bars, footers, sidebars, breadcrumbs, and more. In addition to having lots of pieces, the structure of web pages can be complex so that it's responsive across different platforms and browsers. Much of the source used to manage this complexity gets repeated on most pages across a website. To help with this, the Django Template Engine allows you to define a system of reusable components, called *blocks*.

A template block labels a section of a page with a name using paired `{% block %}` and `{% endblock %}` tags. Template pages can be composed together in an inheritance hierarchy with children gaining the structure of their parent. Any blocks in a child page with the same name as the parent page replace the block in the parent page.

Consider the `<title>` tag in head section of an HTML page. General pages in your site might set the title "RiffMates," while some pages may want to replace this with more specific information. In a parent template, you could declare a block with the name `title`, and then any inheriting child page could override this block with the specific name.

You perform inheritance with the `{% extends %}` tag. A child page declares that it extends a parent and inherits its parent's structure. The most common use of this is to